

Q73. What are the major features introduced in Java 8?

Simple Answer

Java 8 introduced several features that made Java more powerful, concise, and functional.

Some of the major features are:

- Lambda Expressions
- Stream API
- Functional Interfaces
- Method References
- Optional Class
- Default Methods
- Static Methods in Interfaces
- New Date & Time API

Detailed Explanation

Before Java 8, Java required more boilerplate code.

Java 8 introduced functional programming concepts, making code shorter, easier to read, and more maintainable.

These features are widely used in modern frameworks like Spring Boot.

Real-Time Example

Spring Boot applications commonly use:

- Streams for processing collections
- Lambdas for callbacks
- Optional for handling null values
- Method References for cleaner code

Code Example

```
List<String> names = Arrays.asList("Java", "Spring");  
names.forEach(System.out::println);
```

Interview Tip

One of the most common interview questions:

Name five important Java 8 features.

Always mention:

- Lambda
- Stream
- Functional Interface
- Optional
- Method Reference

Key Points

✓ Java 8 introduced Functional Programming.

✓ Reduced boilerplate code.

✓ Improved Collection processing.

✓ Most enterprise applications use Java 8+.

Q74. What is a Lambda Expression?

Simple Answer

A **Lambda Expression** is an anonymous function that provides a concise way to implement a functional interface.

It allows developers to write cleaner and more readable code.

Detailed Explanation

Before Java 8:

```
Runnable r = new Runnable() {  
  
    @Override  
    public void run() {  
  
        System.out.println("Running");  
    }  
};
```

Using Lambda:

```
Runnable r = () -> System.out.println("Running");
```

Much shorter and easier to understand.

Real-Time Example

Spring Boot frequently uses Lambdas with:

- Streams
- Collections
- Event handling
- ExecutorService

Code Example

```
List<String> names = Arrays.asList("Java", "Spring");  
  
names.forEach(name -> System.out.println(name));
```

Interview Tip

Remember:

Lambda works only with **Functional Interfaces**.

Key Points

- ✓ Anonymous function.
- ✓ Reduces boilerplate code.
- ✓ Improves readability.
- ✓ Used with Streams.

Q75. What is a Functional Interface?

Simple Answer

A **Functional Interface** is an interface that contains exactly **one abstract method**.

It can have multiple default and static methods.

Functional Interfaces are the foundation of Lambda Expressions.

Detailed Explanation

Java provides the `@FunctionalInterface` annotation to indicate that an interface is intended to be functional.

Common Functional Interfaces:

- Predicate
- Consumer
- Supplier
- Function
- Runnable
- Comparator

Real-Time Example

Suppose an employee list needs to be filtered.

A `Predicate<Employee>` can be used to check whether an employee belongs to a specific department.

Code Example

```
@FunctionalInterface
interface Greeting {

    void sayHello();
}
```

Lambda implementation:

```
Greeting g = () -> System.out.println("Hello");
```

Interview Tip

Common interview question:

Can a Functional Interface have default methods?

Answer:

Yes.

It can have any number of default and static methods, but only one abstract method.

Key Points

- ✓ One abstract method.
- ✓ Supports Lambda Expressions.
- ✓ Can have default methods.
- ✓ Can have static methods.

Q76. What is the Stream API? What are its advantages?

Simple Answer

The **Stream API** is used to process collections in a functional and declarative manner.

It supports operations such as filtering, mapping, sorting, and reducing data.

Detailed Explanation

Streams do not modify the original collection.

They process data using operations such as:

- filter()
- map()
- sorted()
- distinct()
- limit()
- collect()

Streams make code:

- Cleaner
- Shorter
- Easier to maintain

Real-Time Example

Employee Management System

Retrieve all employees:

- Salary > ₹50,000
- Sort by salary
- Display names

Instead of multiple loops, a Stream can perform the operation in one pipeline.

Code Example

```
List<String> names =  
    Arrays.asList("Java", "Spring", "React");  
  
names.stream()  
    .filter(name -> name.startsWith("S"))  
    .forEach(System.out::println);
```

Output

Spring

Interview Tip

Interview Question:

Does Stream modify the original collection?

Answer:

No.

Streams only process data.

Key Points

- ✓ Functional style.
- ✓ Cleaner code.
- ✓ Original collection remains unchanged.
- ✓ Supports parallel processing.

Q77. What is the difference between `map()` and `filter()` in Streams?

Simple Answer

Both are intermediate Stream operations.

- `filter()` selects elements based on a condition.
- `map()` transforms one element into another.

Detailed Explanation

filter()

Keeps only matching elements.

Example:

```
salary > 50000
```

map()

Transforms data.

Example:

Employee

↓

Employee Name

Real-Time Example

Employee List

Need:

- Employees with salary > ₹50,000
- Display only employee names

Use:

```
filter()
```

↓

```
map()
```

Code Example

```
List<String> names =  
    Arrays.asList("Java", "Spring", "React");  
  
names.stream()  
    .filter(name -> name.length() > 5)  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Output

SPRING

Interview Tip

A favorite interview question:

Which operation removes unwanted data?

Answer:

filter()

Which operation transforms data?

Answer:

map()

Key Points

- ✓ filter() → Selects data.
- ✓ map() → Transforms data.
- ✓ Both are intermediate operations.
- ✓ Commonly used together.

Q78. What is the **reduce ()** method in the Stream API?

Simple Answer

The `reduce ()` method is a terminal operation used to combine all elements of a stream into a single result.

It is commonly used for operations such as:

- Sum
- Product
- Maximum
- Minimum
- Concatenation

Detailed Explanation

Unlike `filter ()` or `map ()`, which return another Stream, `reduce ()` processes all elements and returns a single value.

Syntax:

```
stream.reduce(identity, accumulator);
```

Where:

- **identity** → Initial value.
- **accumulator** → Combines two values.

Real-Time Example

Suppose an employee list contains salaries.

The HR department wants the **total salary expense**.

Instead of using loops, `reduce ()` can calculate the sum efficiently.

Code Example

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);

int sum = numbers.stream()
                  .reduce(0, Integer::sum);

System.out.println(sum);
```

Output

100

Interview Tip

Common interview question:

Which Stream operation returns a single value?

Answer:

`reduce()`

Key Points

- ✓ Terminal operation.
- ✓ Produces one result.
- ✓ Used for aggregation.
- ✓ Commonly used for sum and maximum.

Q79. What is the `collect()` method?

Simple Answer

The `collect()` method is a terminal operation used to convert the elements of a Stream into a collection or another desired result.

It is commonly used to collect Stream results into a `List`, `Set`, or `Map`.

Detailed Explanation

After filtering or transforming data, the Stream needs to be converted back into a collection.

`collect()` performs this operation.

Common collectors:

- `Collectors.toList()`

- `Collectors.toSet()`
- `Collectors.toMap()`
- `Collectors.joining()`
- `Collectors.groupingBy()`

Real-Time Example

Suppose an employee application filters employees whose salary is greater than ₹50,000.

The filtered employees need to be stored in a new list.

Code Example

```
List<String> names =  
    Arrays.asList("Java", "Spring", "React");  
  
List<String> result = names.stream()  
    .filter(name -> name.length() > 5)  
    .collect(Collectors.toList());  
  
System.out.println(result);
```

Output

```
[Spring]
```

Interview Tip

Interview Question:

Which Stream operation converts a Stream into a List?

Answer:

```
collect(Collectors.toList())
```

Key Points

✓ Terminal operation.

✓ Converts Stream into collections.

✓ Frequently used after filtering and mapping.

✓ Uses the `Collectors` utility class.

Q80. What is the `Optional` class? Why is it used?

Simple Answer

`Optional` is a container object introduced in Java 8 that may or may not contain a value.

It helps avoid `NullPointerException` by encouraging explicit handling of missing values.

Detailed Explanation

Before Java 8:

```
Employee emp = getEmployee();
```

```
System.out.println(emp.getName());
```

If `emp` is null, a `NullPointerException` occurs.

Using `Optional`:

```
Optional<Employee> employee = getEmployee();
```

Developers can safely check whether a value is present before using it.

Real-Time Example

Suppose an employee search returns no matching record.

Instead of returning `null`, the method returns `Optional.empty()`.

This forces the caller to handle the absence of a value safely.

Code Example

```
Optional<String> name = Optional.of("Murali");
```

```
System.out.println(name.isPresent());
```

```
System.out.println(name.get());
```

Output

```
true  
Murali
```

Interview Tip

Common interview question:

Does `Optional` replace every `null`?

Answer:

No.

It is mainly used as a **return type** to indicate that a value may be absent.

Key Points

- ✓ Helps avoid `NullPointerException`.
- ✓ Represents optional values.
- ✓ Improves code readability.
- ✓ Commonly used as a method return type.

Q81. What is a Method Reference? How is it different from a Lambda Expression?

Simple Answer

A **Method Reference** is a shorthand syntax for referring to an existing method using the `::` operator.

It makes the code shorter and more readable when a Lambda Expression simply calls an existing method.

Detailed Explanation

Lambda:

```
name -> System.out.println(name)
```

Method Reference:

```
System.out::println
```

Both perform the same task, but Method References are more concise.

Real-Time Example

Displaying employee names from a list.

Instead of writing a Lambda Expression that only prints each name, a Method Reference can directly reference `System.out.println`.

Code Example

```
List<String> names =  
    Arrays.asList("Java", "Spring", "React");  
  
names.forEach(System.out::println);
```

Output

```
Java  
Spring  
React
```

Interview Tip

Method References can refer to:

- Static methods
- Instance methods
- Constructors

The `::` operator is the key syntax to remember.

Key Points

- ✓ Uses the :: operator.
- ✓ Makes code cleaner.
- ✓ Replaces simple Lambda Expressions.
- ✓ Improves readability.

Q82. What are Default and Static Methods in Interfaces?

Simple Answer

Java 8 introduced **default** and **static** methods in interfaces.

This allows interfaces to provide method implementations without breaking existing classes that implement them.

Detailed Explanation

Default Method

A default method has a body and is inherited by implementing classes unless overridden.

Static Method

A static method belongs to the interface itself and is called using the interface name.

Real-Time Example

Suppose a `Payment` interface already exists in a banking application.

A new common method is needed for all payment types.

Instead of modifying every implementing class, a default method can be added to the interface.

Code Example

```
interface Payment {  
  
    default void printReceipt() {  
        System.out.println("Receipt Printed");  
    }  
}
```

```
        static void welcome() {  
            System.out.println("Welcome");  
        }  
    }  
  
    class UpiPayment implements Payment {  
    }  
  
    public class Main {  
        public static void main(String[] args) {  
            UpiPayment payment = new UpiPayment();  
            payment.printReceipt();  
            Payment.welcome();  
        }  
    }
```

Output

```
Receipt Printed  
Welcome
```

Interview Tip

A common interview question:

Why were default methods introduced?

Answer:

To allow new functionality to be added to interfaces **without breaking existing implementations**, ensuring backward compatibility.

Key Points

- ✓ Introduced in Java 8.
- ✓ Default methods provide implementation.
- ✓ Static methods belong to the interface.

✓ Improve backward compatibility.